

Definição

Assim como acontece com muitos termos emergentes neste campo de ritmo acelerado, a definição de "desenvolvimento orientado a especificações" (SDD) ainda está em constante mudança. Eis o que pude inferir de como tenho visto o termo ser usado até agora: Desenvolvimento orientado a especificações significa escrever uma "especificação" antes de escrever o código com IA ("documentação primeiro"). A especificação se torna a fonte de verdade tanto para o humano quanto para a IA. GitHub : "Neste novo mundo, manter software significa evoluir as especificações . [...] A linguagem comum do desenvolvimento passa para um nível superior, e o código é a abordagem final."

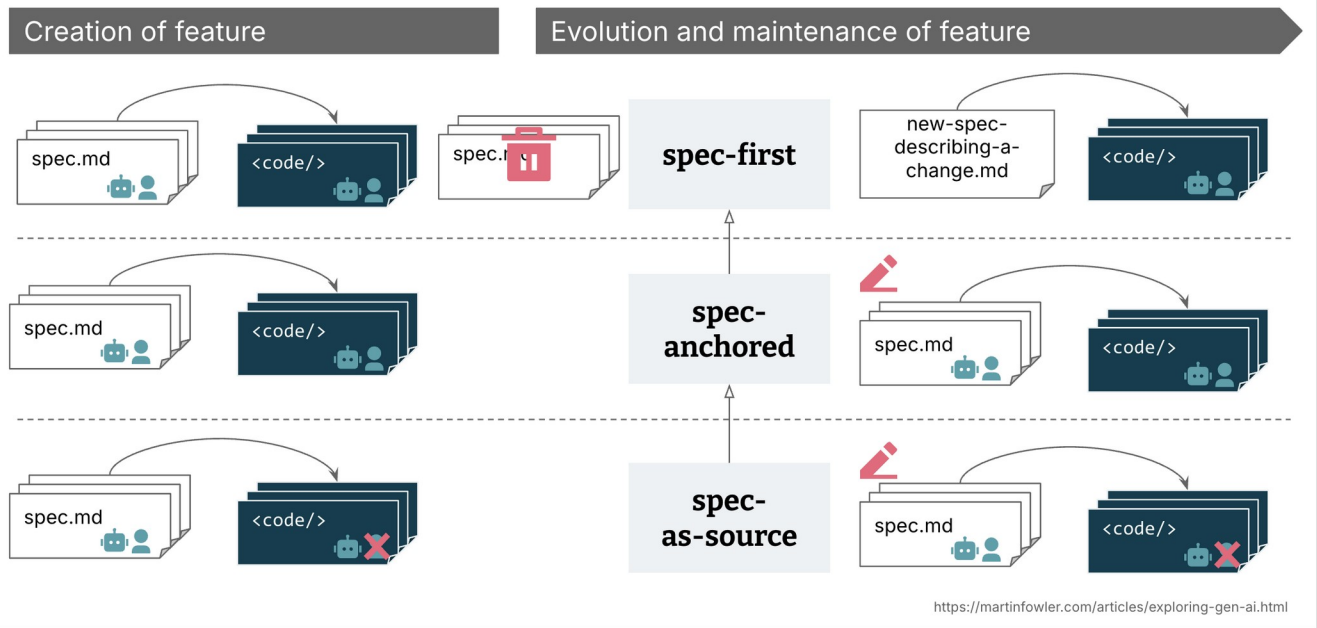
Tessl : "Uma abordagem de desenvolvimento onde as especificações — e não o código — são o principal artefato . As especificações descrevem a intenção em uma linguagem estruturada e testável, e os agentes geram código para corresponder a elas."

Após analisar os usos do termo e algumas das ferramentas que afirmam implementar o SDD (Software Development Design), parece-me que, na realidade, existem vários níveis de implementação:

- 1.Especificação em primeiro lugar : Uma especificação bem elaborada é escrita primeiro e, em seguida, usada no fluxo de trabalho de desenvolvimento assistido por IA para a tarefa em questão.
- 2.Ancorado na especificação : A especificação é mantida mesmo após a conclusão da tarefa, para continuar sendo usada na evolução e manutenção do respectivo recurso.
- 3.Especificação como fonte : A especificação é o arquivo fonte principal ao longo do tempo, e somente a especificação é editada pelo usuário; o usuário nunca toca no código.

Todas as abordagens e definições de SDD que encontrei priorizam a especificação, mas nem todas buscam ser ancoradas na especificação ou considerá-la como fonte. Além disso, muitas vezes a estratégia de manutenção da especificação ao longo do tempo fica vaga ou totalmente em aberto.

Levels of “spec-driven”



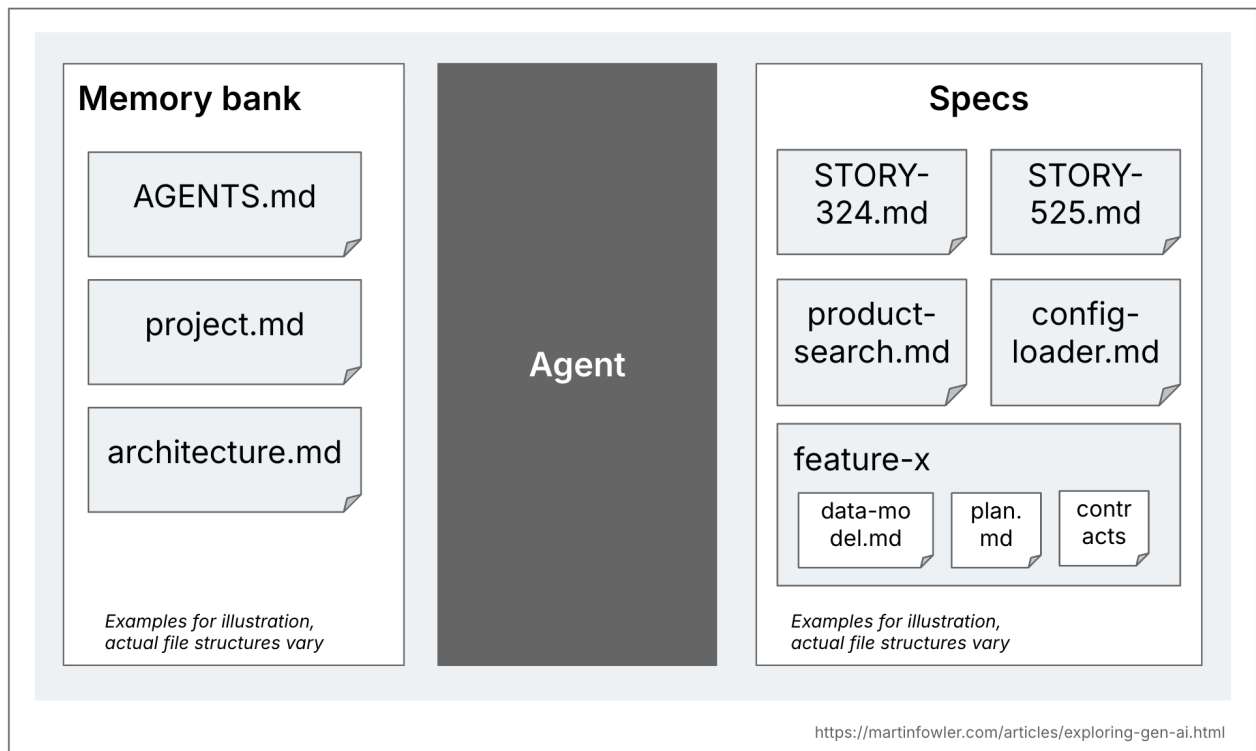
O que é uma especificação?

A questão fundamental em termos de definições é, obviamente, o que é uma especificação? Não parece haver uma definição geral; a mais próxima que encontrei de uma definição consistente é a comparação de uma especificação com um "Documento de Requisitos do Produto".

O termo está bastante sobrecarregado no momento; aqui está minha tentativa de definir o que é uma especificação:

Uma especificação é um artefato estruturado e orientado a comportamento — ou um conjunto de artefatos relacionados — escrito em linguagem natural que expressa a funcionalidade do software e serve como guia para agentes de codificação de IA. Cada variante do desenvolvimento orientado a especificações define sua abordagem para a estrutura da especificação, o nível de detalhamento e como esses artefatos são organizados dentro de um projeto.

Acredito que exista uma diferença importante a ser feita entre as especificações e os documentos de contexto mais gerais de uma base de código. Esse contexto geral inclui arquivos de regras ou descrições de alto nível do produto e da base de código. Algumas ferramentas chamam esse contexto de banco de memória, e é esse termo que usarei aqui. Esses arquivos são relevantes para todas as sessões de codificação de IA na base de código, enquanto as especificações são relevantes apenas para as tarefas que de fato criam ou alteram essa funcionalidade específica.



O desafio na avaliação de ferramentas de SDD

Avaliar ferramentas e abordagens de SDD (Software Development Development) de uma forma que se aproxime do uso real acaba sendo bastante demorado. Seria necessário testá-las com problemas de diferentes tamanhos, em projetos novos e existentes, e dedicar tempo para revisar e aprimorar os artefatos intermediários com mais profundidade do que uma simples olhada superficial. Afinal, como diz a postagem do GitHub sobre o spec-kit : "Fundamentalmente, seu papel não é apenas direcionar. É verificar. Em cada fase, você reflete e refina."

Para duas das três ferramentas que testei, parece que integrá-las a uma base de código existente exige ainda mais trabalho, o que dificulta ainda mais a avaliação de sua utilidade em bases de código já existentes. Até que eu receba relatos de usuários que as utilizem por um período em uma base de código "real", ainda tenho muitas dúvidas sobre como isso funciona na prática.

Dito isso, vamos analisar três dessas ferramentas. Primeiro, compartilharei uma descrição de como elas funcionam (ou melhor, como eu acho que funcionam) e, por fim, apresentarei minhas observações e perguntas. Observe que essas ferramentas estão em constante evolução, portanto, podem já ter sofrido alterações desde que as utilizei em setembro.

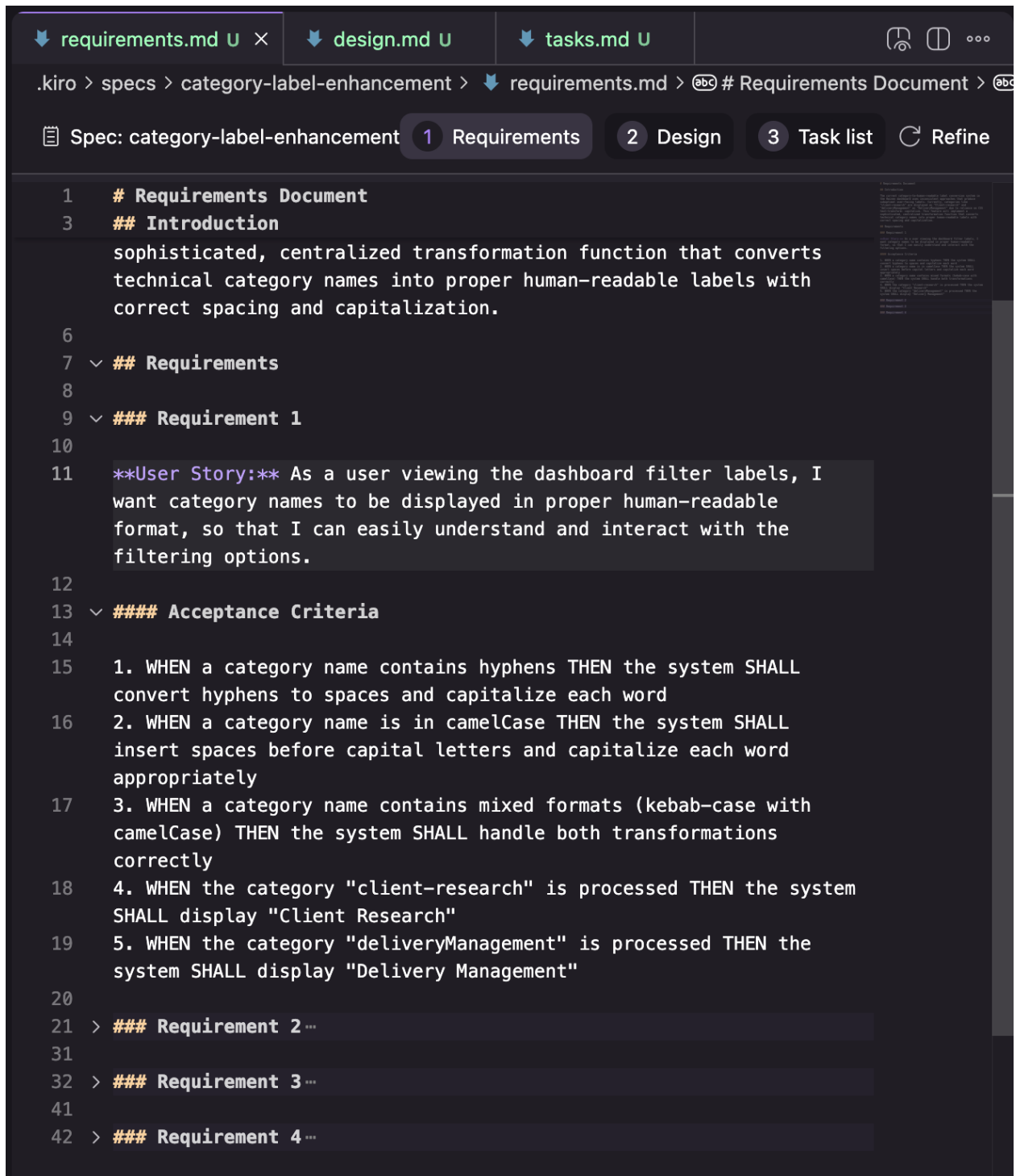
Kiro

O Kiro é o mais simples (ou mais leve) dos três que experimentei. Parece ser focado principalmente na especificação; todos os exemplos que encontrei o utilizam para

uma tarefa ou uma história de usuário, sem mencionar como usar o documento de requisitos de forma ancorada na especificação ao longo do tempo, em várias tarefas. Fluxo de trabalho: Requisitos → Projeto → Tarefas

Cada etapa do fluxo de trabalho é representada por um documento Markdown, e o Kiro guia você por essas 3 etapas dentro de sua distribuição baseada no VS Code.

Requisitos: Estruturados como uma lista de requisitos, onde cada requisito representa uma “História de Usuário” (no formato “Como um...” com critérios de aceitação (no formato “DADO... QUANDO... ENTÃO...”).



The screenshot shows the Kiro application interface within a VS Code environment. At the top, there are three tabs: 'requirements.md U', 'design.md U', and 'tasks.md U'. Below the tabs, the breadcrumb path is '.kiro > specs > category-label-enhancement > requirements.md > # Requirements Document'. A navigation bar below the breadcrumb shows three steps: '1 Requirements' (active), '2 Design', and '3 Task list', followed by a 'Refine' button. The main editor area displays a Markdown document with the following content:

```
1 # Requirements Document
3 ## Introduction
sophisticated, centralized transformation function that converts
technical category names into proper human-readable labels with
correct spacing and capitalization.
6
7 ∨ ## Requirements
8
9 ∨ ### Requirement 1
10
11 **User Story:** As a user viewing the dashboard filter labels, I
want category names to be displayed in proper human-readable
format, so that I can easily understand and interact with the
filtering options.
12
13 ∨ #### Acceptance Criteria
14
15 1. WHEN a category name contains hyphens THEN the system SHALL
convert hyphens to spaces and capitalize each word
16 2. WHEN a category name is in camelCase THEN the system SHALL
insert spaces before capital letters and capitalize each word
appropriately
17 3. WHEN a category name contains mixed formats (kebab-case with
camelCase) THEN the system SHALL handle both transformations
correctly
18 4. WHEN the category "client-research" is processed THEN the system
SHALL display "Client Research"
19 5. WHEN the category "deliveryManagement" is processed THEN the
system SHALL display "Delivery Management"
20
21 > ### Requirement 2 ...
31
32 > ### Requirement 3 ...
41
42 > ### Requirement 4 ...
```

Design: Na minha tentativa, o documento de design consistiu nas seções mostradas na captura de tela abaixo. Só tenho os resultados de uma das minhas tentativas, então não sei se essa é uma estrutura consistente ou se ela muda dependendo da tarefa.

requirements.md U design.md U x tasks.md U

.kiri > specs > category-label-enhancement > design.md > # Design Document >

Spec: category-label-enhancement 1 Requirements 2 Design 3 Task list

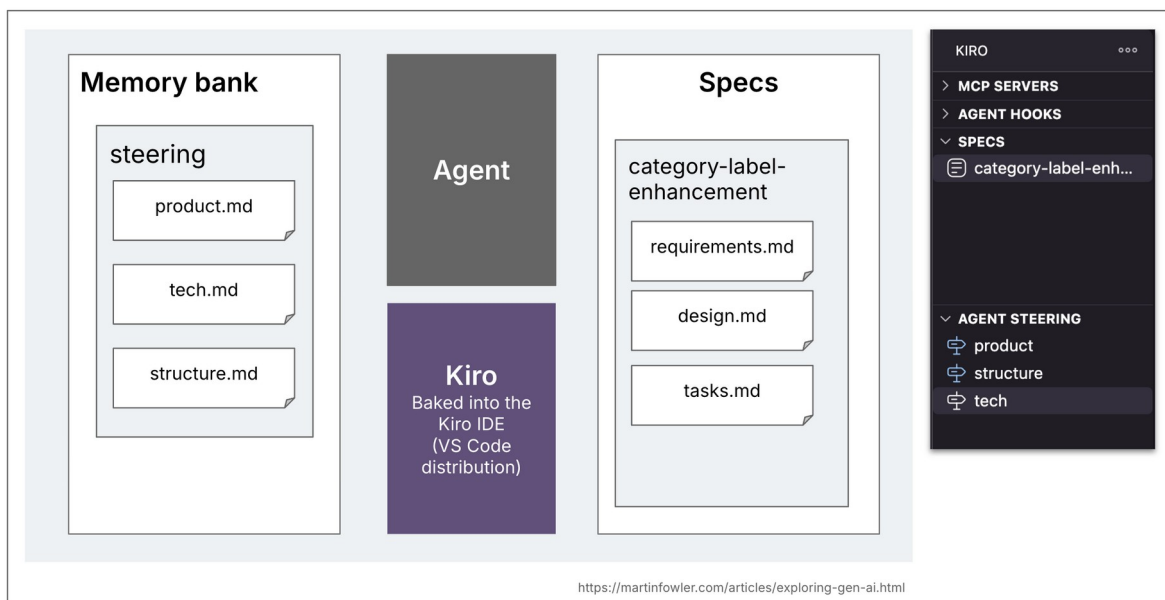
```
1  # Design Document
10
11 ## Architecture
12
13 ### Component Architecture
14
15 ```
16
17     UI Components
18
19     | Dashboard | Sidebar | Card Categories |
20     | Filter Tags | Navigation | Tags |
21
22     | | | | | | | | | | | | |
23     | | | | | | | | | | | | |
24
25     categoryLabelUtils.js
26
27     |
28     | toCategoryLabel(category)
29
30     | Input: "client-research", "deliveryManagement"
31     | Output: "Client Research", "Delivery Management"
32
33
34 ```
35
36 > ### Data Flow ...
41
42 ## Components and Interfaces
43
44 > ### Core Utility Function ...
58
59 > ### Integration Points ...
79
80 > ## Data Models ...
102
103 > ## Error Handling ...
115
116 > ## Testing Strategy ...
163
164 > ## Implementation Approach ...
185
186 > ## Migration Strategy ...
```

Tarefas: Uma lista de tarefas que remontam aos números dos requisitos e que recebem alguns elementos de interface do usuário adicionais para executar as tarefas uma a uma e revisar as alterações em cada tarefa.

The screenshot shows the Kiro IDE interface with a dark theme. At the top, there are tabs for 'requirements.md U', 'design.md U', and 'tasks.md U'. Below the tabs, the breadcrumb path is '.kiro > specs > category-label-enhancement > tasks.md > @ # Implementation Plan'. The main area is divided into three sections: 'Spec: category-label-enhancement', 'Requirements', 'Design', and 'Task list'. The 'Task list' section is active, showing a list of tasks. The first task is '1. Create core utility function and test suite', which is expanded to show sub-tasks: 'Create `ui/src/app/Utils/categoryLabelUtils.js` with `toCategoryLabel()` function', 'Implement transformation logic for kebab-case, camelCase, snake_case, and mixed formats', 'Handle edge cases including null/undefined inputs, empty strings, and special characters', and 'Requirements: 1.1, 1.2, 1.3, 4.1, 4.2, 4.3_'. The second task is '1.1 Write comprehensive test suite for utility function', which is also expanded to show sub-tasks: 'Create `ui/src/\_\_tests\_\_/utils/categoryLabelUtils.test.js`', 'Write unit tests for basic transformations (kebab-case, camelCase, snake_case)', 'Write tests for complex cases (mixed formats, multiple words, numbers)', 'Write tests for edge cases (null, undefined, empty strings, already formatted)', 'Ensure test coverage for specific examples: "client-research" -> "Client Research", "deliveryManagement" -> "Delivery Management"', and 'Requirements: 1.4, 1.5, 4.1, 4.2, 4.3, 4.4_'. The third task is '2. Update dashboard filter tags implementation', which is partially visible.

```
1 # Implementation Plan
2
3 Task in progress | View changes
4 [-] 1. Create core utility function and test suite
5   - Create `ui/src/app/Utils/categoryLabelUtils.js` with `toCategoryLabel()` function
6   - Implement transformation logic for kebab-case, camelCase, snake_case, and mixed formats
7   - Handle edge cases including null/undefined inputs, empty strings, and special characters
8   - Requirements: 1.1, 1.2, 1.3, 4.1, 4.2, 4.3_
9
10 [-] 1.1 Write comprehensive test suite for utility function
11   - Create `ui/src/__tests__/utils/categoryLabelUtils.test.js`
12   - Write unit tests for basic transformations (kebab-case, camelCase, snake_case)
13   - Write tests for complex cases (mixed formats, multiple words, numbers)
14   - Write tests for edge cases (null, undefined, empty strings, already formatted)
15   - Ensure test coverage for specific examples: "client-research" -> "Client Research",
16     "deliveryManagement" -> "Delivery Management"
17   - Requirements: 1.4, 1.5, 4.1, 4.2, 4.3, 4.4_
18
19 Start task
20 [-] 2. Update dashboard filter tags implementation
21   - Modify `ui/src/pages/index.js` to import and use `toCategoryLabel()` function
```

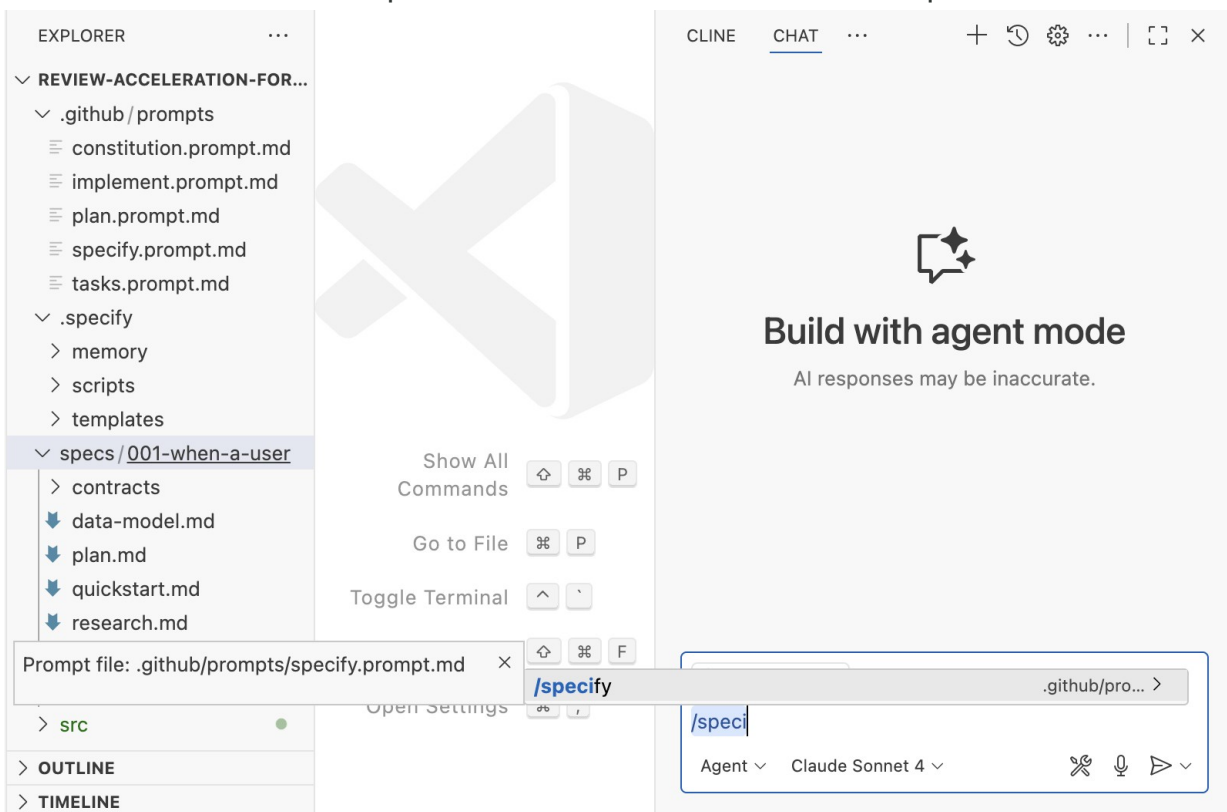
O Kiro também possui o conceito de um banco de memória, que eles chamam de "direção". Seu conteúdo é flexível e seu fluxo de trabalho não parece depender da presença de arquivos específicos (fiz meus testes de uso antes mesmo de descobrir a seção de direção). A topologia padrão criada pelo Kiro quando você solicita a geração de documentos de direção é product.md, structure.md, tech.md.



Kit de especificações

O Spec-kit é a versão do GitHub do SDD. Ele é distribuído como uma CLI que pode criar configurações de espaço de trabalho para uma ampla gama de assistentes de codificação comuns. Uma vez que essa estrutura esteja configurada, você interage com o Spec-kit por meio de comandos de barra no seu assistente de codificação.

Como todos os seus artefatos são colocados diretamente no seu espaço de trabalho, esta é a ferramenta mais personalizável das três discutidas aqui.



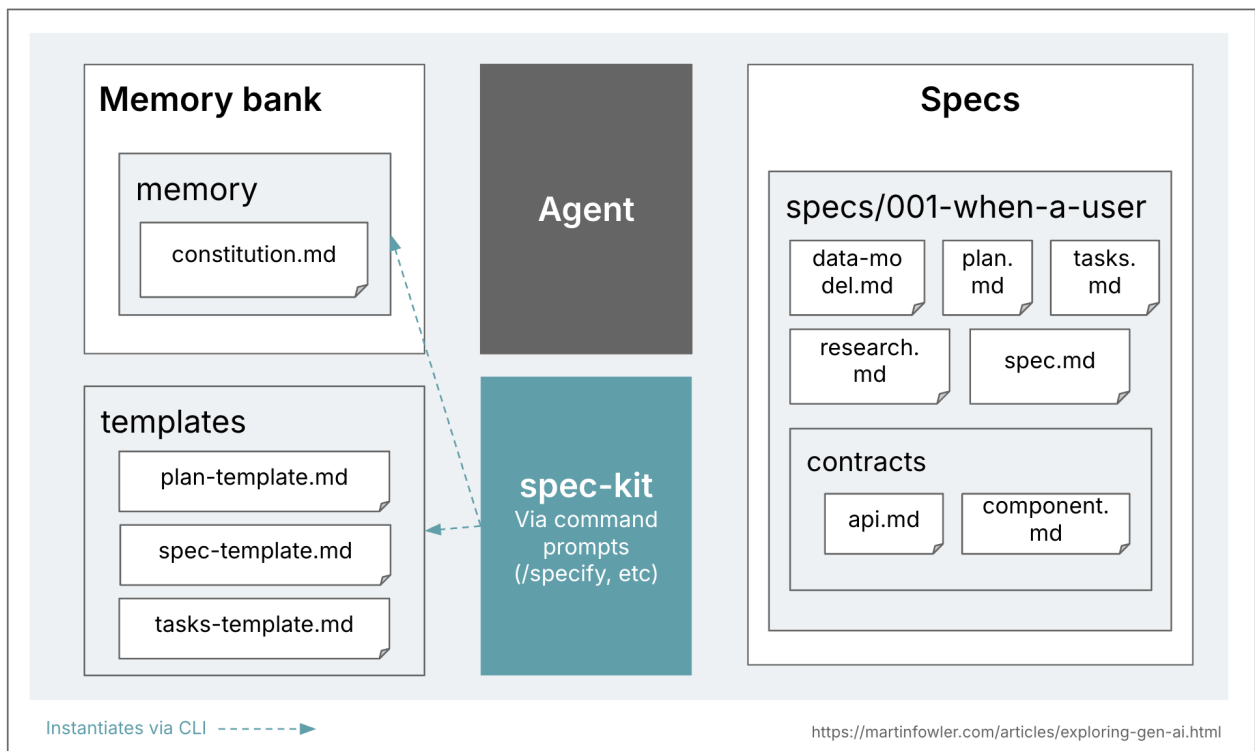
Fluxo de trabalho: Constituição → Especificar → Planejar → Tarefas

O conceito de banco de memória do Spec-kit é um pré-requisito para a abordagem orientada a especificações. Eles o chamam de constituição . A constituição deve conter os princípios de alto nível que são "imutáveis" e devem ser sempre aplicados a cada alteração. É basicamente um arquivo de regras muito poderoso, amplamente utilizado no fluxo de trabalho.

Em cada uma das etapas do fluxo de trabalho (especificar, planejar, tarefas), o spec-kit instancia um conjunto de arquivos e prompts com a ajuda de um script bash e alguns modelos. O fluxo de trabalho utiliza amplamente listas de verificação dentro dos arquivos para acompanhar esclarecimentos necessários do usuário, violações de regras, tarefas de pesquisa, etc. Elas funcionam como uma "definição de conclusão" para cada etapa do fluxo de trabalho (embora interpretadas por IA, portanto não há garantia de 100% de que serão respeitadas).

```
spec.md × constitution.md plan.md
specs > 001-when-a-user > spec.md > # Feature Specification: Dashboard Change Impact Summary >
1  # Feature Specification: Dashboard Change Impact Summary
59 ## Requirements *(mandatory)*
72 ### Key Entities *(include if feature involves data)*
73
80
81 ## Review & Acceptance Checklist
82 *GATE: Automated checks run during main() execution*
83
84 ### Content Quality
85 - [x] No implementation details (languages, frameworks, APIs)
86 - [x] Focused on user value and business needs
87 - [x] Written for non-technical stakeholders
88 - [x] All mandatory sections completed
89
90 ### Requirement Completeness
91 - [x] No [NEEDS CLARIFICATION] markers remain
92 - [x] Requirements are testable and unambiguous
93 - [x] Success criteria are measurable
94 - [x] Scope is clearly bounded
95 - [x] Dependencies and assumptions identified
96
97 ---
98
99 ## Execution Status
100 *Updated by main() during processing*
101
102 - [x] User description parsed
103 - [x] Key concepts extracted
104 - [x] Ambiguities marked
105 - [x] User scenarios defined
106 - [x] Requirements generated
107 - [x] Entities identified
108 - [x] Review checklist passed
109
110 ---
111
```

A seguir, apresento uma visão geral para ilustrar a topologia de arquivos que observei no spec-kit. Observe como uma especificação é composta por vários arquivos.

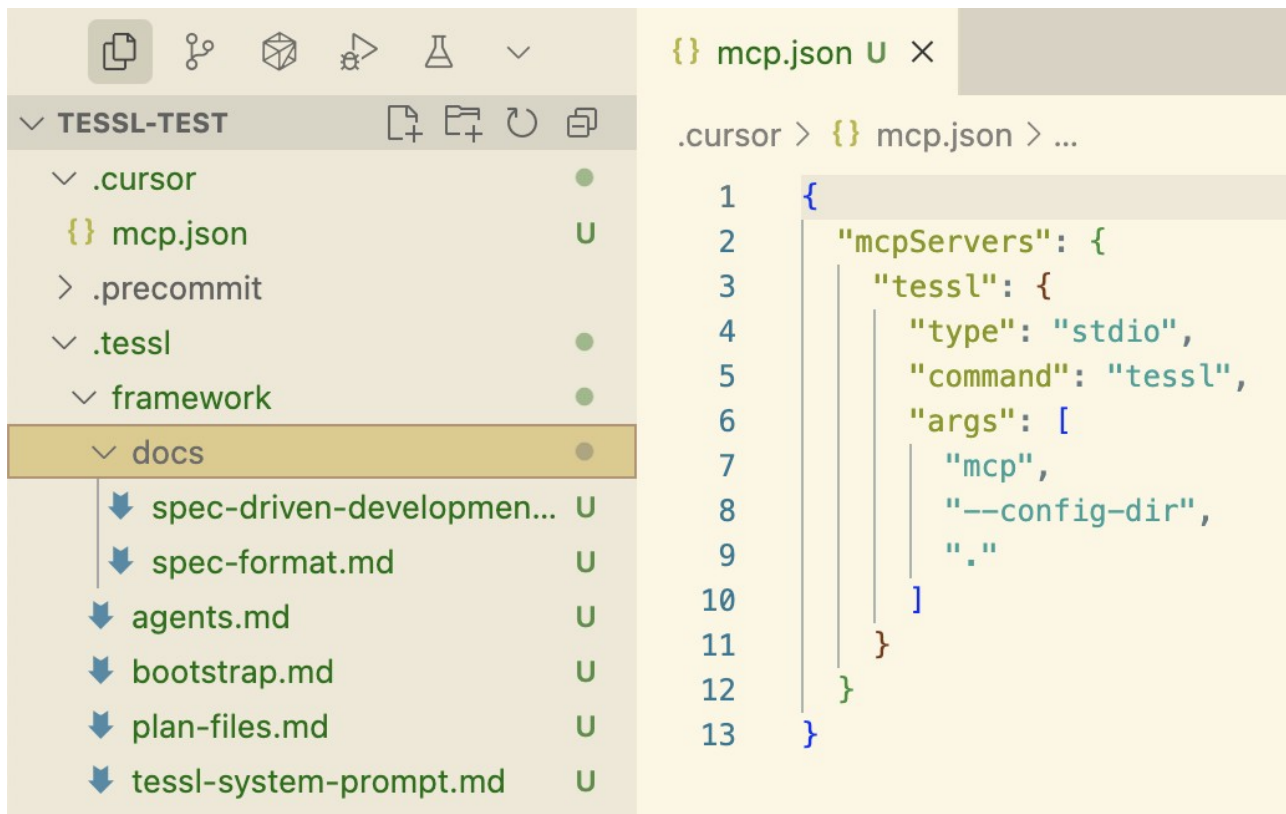


À primeira vista, o GitHub parece aspirar a uma abordagem ancorada em especificações ("É por isso que estamos repensando as especificações — não como documentos estáticos, mas como artefatos vivos e executáveis que evoluem com o projeto. As especificações se tornam a fonte de verdade compartilhada. Quando algo não faz sentido, você volta à especificação; quando um projeto se torna complexo, você o refina; quando as tarefas parecem muito grandes, você as divide."). No entanto, o spec-kit cria uma branch para cada especificação criada, o que parece indicar que eles veem uma especificação como um artefato vivo durante o ciclo de vida de uma solicitação de alteração, e não durante o ciclo de vida de uma funcionalidade. Esta discussão na comunidade aborda essa confusão. Isso me leva a crer que o spec-kit ainda é o que eu chamaria de "prioridade à especificação", e não ancorado em especificações ao longo do tempo.

Framework Tessler

(Ainda em beta privado)

Assim como o spec-kit, o Tessler Framework é distribuído como uma CLI que pode criar toda a estrutura de espaço de trabalho e configuração para uma variedade de assistentes de codificação. O comando da CLI também funciona como um servidor MCP.



Dentre essas três ferramentas, o Tessl é a única que aspira explicitamente a uma abordagem ancorada em especificações, chegando a explorar o nível de especificação como fonte no Desenvolvimento Orientado a Software (SDD). Uma especificação Tessl pode servir como o artefato principal que está sendo mantido e editado, com o código inclusive marcado com um comentário no topo `// GENERATED FROM SPEC - DO NOT EDIT`. Atualmente, existe um mapeamento 1:1 entre os arquivos de especificação e código, ou seja, uma especificação corresponde a um arquivo no código-fonte. No entanto, o Tessl ainda está em versão beta e a equipe está experimentando diferentes versões disso, então imagino que essa abordagem também possa ser adotada em um nível onde uma especificação corresponde a um componente de código com múltiplos arquivos. Resta saber o que a versão alfa suportará. (A própria equipe do Tessl considera seu framework como algo mais voltado para o futuro do que seu produto público atual, o Registro Tessl.) Aqui está um exemplo de uma especificação que eu pedi para o Tessl CLI realizar engenharia reversa (`tessl document --code ...js`) a partir de um arquivo JavaScript em uma base de código existente:

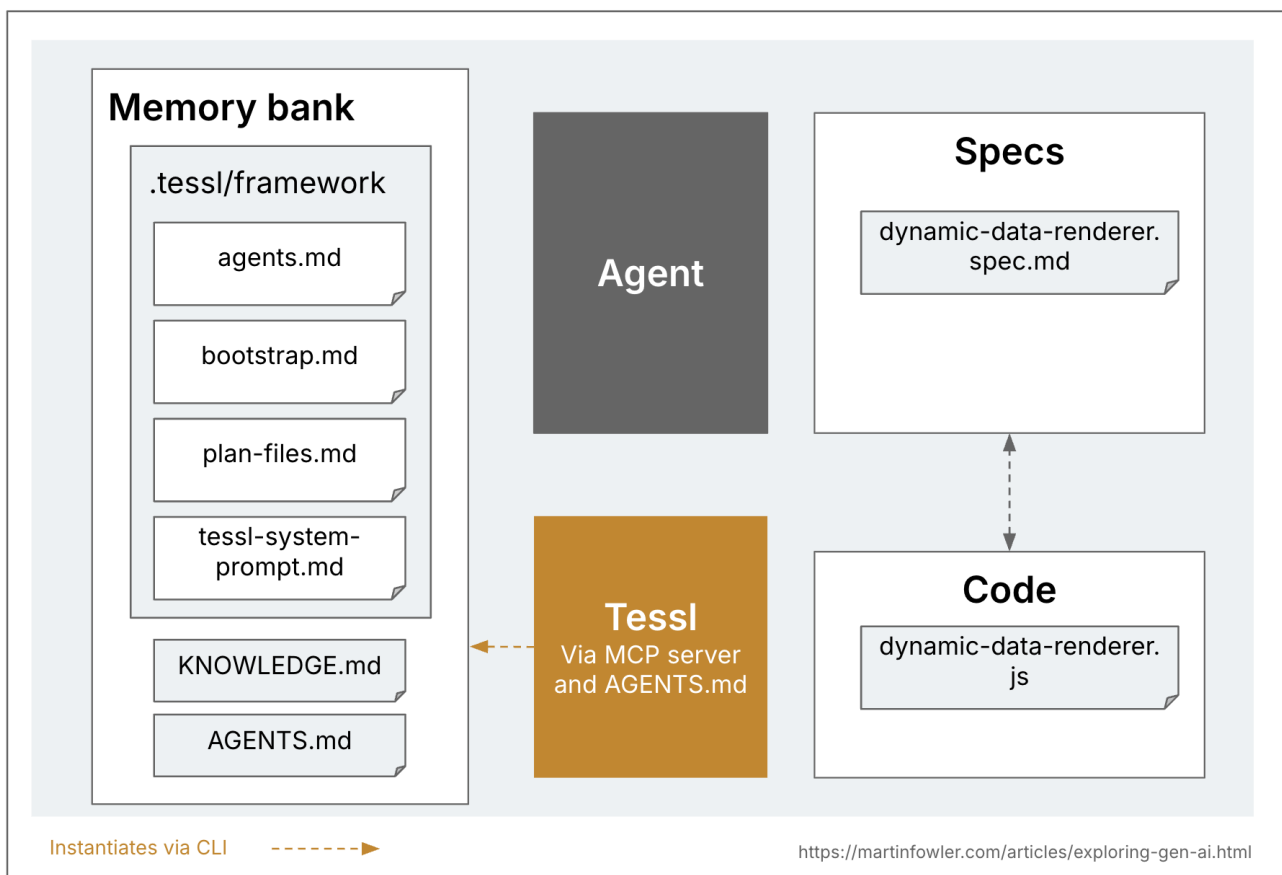
```

↓ _dynamic_data_renderer.spec.md U × JS _dynamic_data_renderer.js 9, M
ui > src > app > ↓ _dynamic_data_renderer.spec.md > [abc] # Dynamic Data Renderer > [abc] ## API
1  # Dynamic Data Renderer
2
3  A React component system for dynamically rendering complex data objects with intelligent type
  detection and formatting.
4
5  ## Target
6
7  [@generate](./_dynamic_data_renderer.js)
8
9  ## Capabilities
10
11  ### Convert keys to readable text
12
13  Transforms various naming conventions into human-readable labels.
14
15  - Converts snake_case to "Snake case" [@test](./tests/_dynamic_data_renderer.test.js)
16  - Converts kebab-case to "Kebab case" [@test](./tests/_dynamic_data_renderer.test.js)
17  - Converts camelCase to "Camel case" [@test](./tests/_dynamic_data_renderer.test.js)
18  - Capitalizes first letter of converted text [@test](./tests/_dynamic_data_renderer.test.js)
19  - Trims whitespace from converted text [@test](./tests/_dynamic_data_renderer.test.js)
20
21 > ### Convert scenario data to markdown...
31
32 > ### Render dynamic data components...
45
46 > ### Render different value types...
58
59  ## API
60
61  ```javascript { .api }
62  // Helper function to convert naming conventions to readable text
63  export const toReadableText = (key: string) => string;
64
65  // Convert scenario data to markdown format
66  export const scenarioToText = (
67    data: object,
68    exclude?: string[] | number,
69    headerLevel?: number
70  ) => string;
71
72  // Main component for dynamic data rendering
73  export const DynamicDataRenderer = ({
74    data: any,
75    exclude?: string[],
76    skipTitles?: boolean,
77    className?: string
78  }) => React.Component;
79
80  // Helper to render different value types
81  export const renderValue = (value: any, parentKey?: string) => React.Component;

```

Tags como `@Component` `@generate` ou `@testComponent` parecem indicar ao Tessler o que gerar. A seção da API demonstra a ideia de definir pelo menos as interfaces que serão expostas a outras partes do código-fonte na especificação, presumivelmente para garantir que essas partes mais cruciais do componente

gerado estejam totalmente sob o controle do mantenedor. Executar ``tessl run` tessl build` para esta especificação gera o arquivo de código JavaScript correspondente. Ao definir as especificações para o código-fonte em um nível de abstração bastante baixo, por arquivo de código, provavelmente reduz a quantidade de etapas e interpretações que o LLM precisa realizar e, portanto, a probabilidade de erros. Mesmo nesse baixo nível de abstração, observei o não determinismo em ação ao gerar código várias vezes a partir da mesma especificação. Foi um exercício interessante iterar sobre a especificação e torná-la cada vez mais específica para aumentar a repetibilidade da geração de código. Esse processo me lembrou algumas das armadilhas e desafios de escrever uma especificação completa e inequívoca.



Observações e perguntas

Essas três ferramentas se autodenominam implementações de desenvolvimento orientado a especificações (SDD), mas são bastante diferentes entre si. Portanto, esse é o primeiro ponto a se ter em mente ao falar sobre SDD: não se trata de uma única coisa.

Um fluxo de trabalho que se adapta a todos os tamanhos?

Kiro e spec-kit oferecem fluxos de trabalho com opiniões bem definidas, mas tenho quase certeza de que nenhum deles é adequado para a maioria dos problemas de programação da vida real. Em particular, não me parece claro como eles atenderiam

a uma variedade suficiente de tamanhos de problemas para serem aplicáveis de forma geral.

Quando pedi à Kiro para corrigir um pequeno bug (o mesmo que usei no passado para testar o Codex), ficou claro rapidamente que o fluxo de trabalho era como usar uma marreta para quebrar uma noz. O documento de requisitos transformou esse pequeno bug em 4 "histórias de usuário" com um total de 16 critérios de aceitação, incluindo pérolas como "História de usuário: Como desenvolvedor, quero que a função de transformação lide com casos extremos de forma adequada, para que o sistema permaneça robusto quando novos formatos de categoria forem introduzidos."

Enfrentei um desafio semelhante ao usar o Speckit. Não tinha certeza sobre o tamanho do problema para o qual deveria utilizá-lo. Os tutoriais disponíveis geralmente se baseiam na criação de um aplicativo do zero, pois isso facilita o processo. Um dos casos de uso que acabei testando foi uma funcionalidade que representaria uma tarefa de 3 a 5 pontos em uma das minhas equipes anteriores. Essa funcionalidade dependia de muito código já existente e deveria criar um modal de visão geral que resumisse diversos dados de um painel de controle existente. Com a quantidade de etapas que o Speckit exigiu e a quantidade de arquivos Markdown que ele gerou para minha revisão, senti que era um exagero para o tamanho do problema. Era um problema maior do que o que usei com o Kiro, mas também um fluxo de trabalho muito mais complexo. Nem cheguei a concluir a implementação completa, mas acredito que, no mesmo tempo que levei para executar e revisar os resultados do Speckit, eu poderia ter implementado a funcionalidade com programação assistida por IA "pura" e me sentiria muito mais no controle. Uma ferramenta SDD eficaz teria, no mínimo, que oferecer flexibilidade para alguns fluxos de trabalho principais diferentes, para alterações de tamanhos e tipos variados.

Revisar Markdown em vez de revisar código?

Como mencionei, e como você pode ver na descrição da ferramenta acima, o Speckit gerou MUITOS arquivos Markdown para eu revisar. Eles eram repetitivos, tanto entre si quanto em relação ao código já existente. Alguns já continham código. No geral, eram muito verbosos e tediosos de revisar. No Kiro, foi um pouco mais fácil, pois você só recebe 3 arquivos, e é mais intuitivo entender o modelo mental de "requisitos > design > tarefas". No entanto, como mencionei, o Kiro também era verboso demais para o pequeno bug que eu precisava corrigir.

Sinceramente, prefiro revisar código do que todos esses arquivos Markdown. Uma ferramenta de desenvolvimento de software eficaz precisaria proporcionar uma ótima experiência de revisão de especificações.

Falsa sensação de controle?

Mesmo com todos esses arquivos, modelos, instruções, fluxos de trabalho e listas de verificação, frequentemente observei o agente não seguir todas as instruções. Sim, as janelas de contexto agora são maiores, o que é frequentemente mencionado como um dos facilitadores do desenvolvimento orientado a especificações. Mas só porque as janelas são maiores, não significa que a IA irá captar tudo o que está ali. Por exemplo: o Spec-kit tem uma etapa de pesquisa em algum ponto do planejamento, e realizou uma extensa pesquisa sobre o código existente e o que já estava lá, o que foi ótimo porque eu pedi para adicionar um recurso que se baseasse no código existente. Mas, no fim das contas, o agente ignorou as observações de que essas eram descrições de classes existentes, simplesmente as interpretou como uma nova especificação e as gerou novamente, criando duplicatas. Mas eu não vi apenas exemplos de instruções ignoradas; também vi o agente exagerar por seguir instruções com muita zelo (como, por exemplo, um dos artigos da constituição). O passado mostrou que a melhor maneira de mantermos o controle sobre o que estamos construindo é por meio de etapas pequenas e iterativas. Portanto, sou muito cético quanto à ideia de que um projeto de especificação detalhado e inicial seja uma boa ideia, especialmente quando se torna excessivamente prolixo. Uma ferramenta de Desenvolvimento de Software (SDD) eficaz precisaria se adequar a uma abordagem iterativa, mas pequenos pacotes de trabalho parecem quase contradizer a própria ideia de SDD.

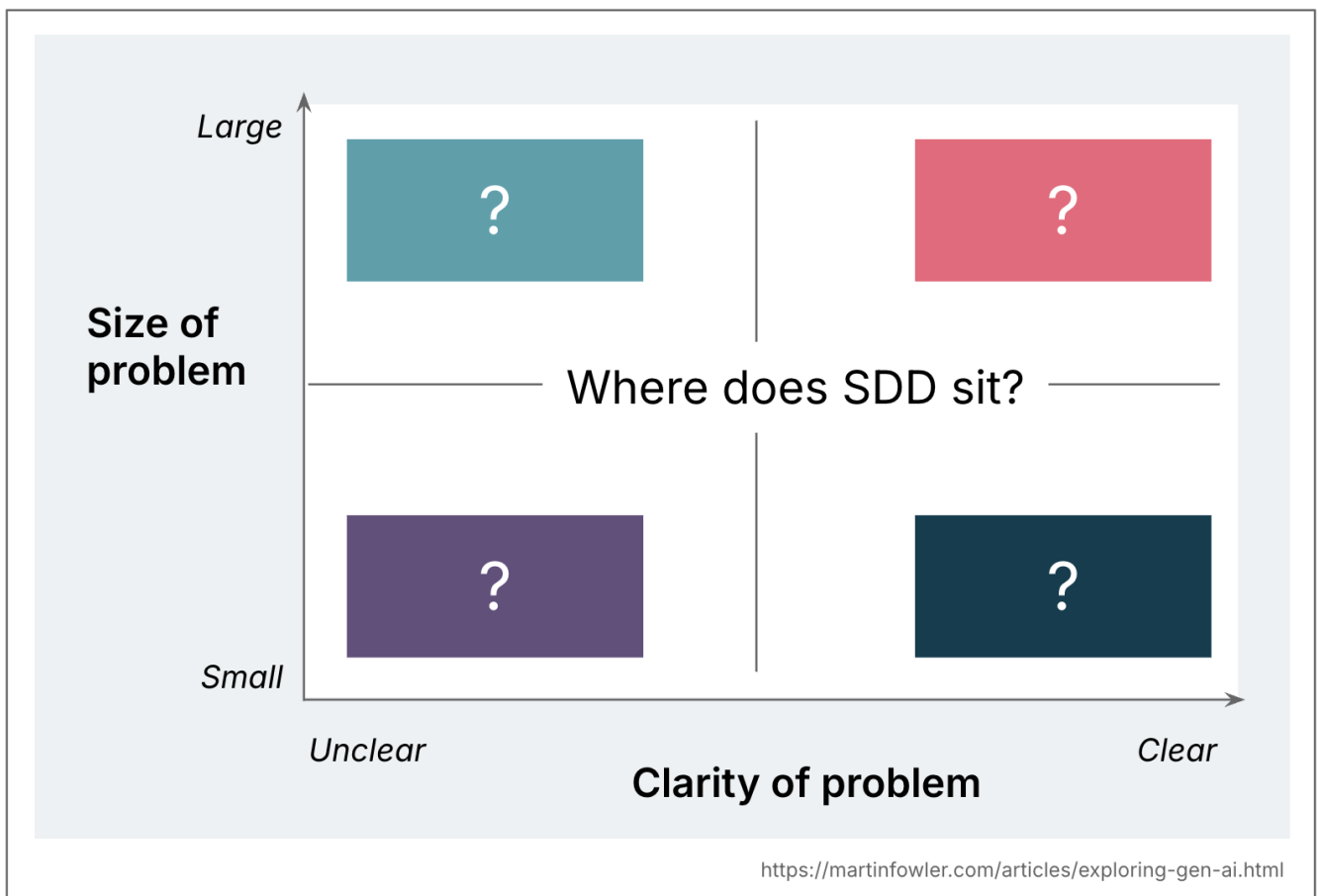
Como separar efetivamente as especificações funcionais das especificações técnicas?

É uma ideia comum em SDD (Software Development Development) buscar intencionalmente a separação entre a especificação funcional e a implementação técnica. A aspiração subjacente, creio eu, é que, em última análise, possamos ter IA (Inteligência Artificial) para preencher todas as soluções e detalhes, e alternar entre diferentes conjuntos de tecnologias com a mesma especificação.

Na realidade, quando eu estava experimentando o SpecKit, frequentemente me confundia sobre quando me manter no nível funcional e quando era hora de adicionar detalhes técnicos. O tutorial e a documentação também não eram muito consistentes nesse sentido; parecia haver diferentes interpretações do que realmente significa "puramente funcional". E quando penso nas inúmeras histórias de usuário que li ao longo da minha carreira que não separavam adequadamente os requisitos da implementação, acho que não temos um bom histórico como profissão nesse quesito.

Quem é o usuário-alvo?

Muitas das demonstrações e tutoriais de ferramentas de desenvolvimento orientadas a especificações incluem aspectos como a definição de objetivos de produto e funcionalidades, e até mesmo incorporam termos como "história de usuário". A ideia aqui seria usar IA como um facilitador para o desenvolvimento de habilidades transversais, permitindo que os desenvolvedores participem mais ativamente da análise de requisitos? Ou que trabalhem em conjunto com a equipe de produto nesse fluxo de trabalho? Nada disso é explicitado, porém; é apresentado como algo dado como certo que um desenvolvedor realizará toda essa análise. Nesse caso, eu me perguntaria novamente: para qual tamanho e tipo de problema o SDD foi projetado? Provavelmente não para funcionalidades grandes e ainda muito obscuras, pois isso certamente exigiria habilidades mais especializadas em produto e requisitos, além de várias outras etapas, como pesquisa e envolvimento das partes interessadas.



Especificações ancoradas e especificações como fonte: estamos aprendendo com o passado?

Embora muitas pessoas façam analogias entre SDD e TDD ou BDD, acredito que outro paralelo importante a ser considerado, especialmente para a especificação como fonte, seja o MDD (desenvolvimento orientado a modelos). Trabalhei em

alguns projetos no início da minha carreira que utilizavam bastante MDD, e isso me veio à mente constantemente quando experimentei o framework Tessel. Os modelos em MDD eram basicamente as especificações, embora não em linguagem natural, mas expressas, por exemplo, em UML personalizado ou uma DSL textual. Criávamos geradores de código personalizados para transformar essas especificações em código.

```
Screen "Write Message" instantiates InputScreen {  
    Title "XYZ";  
    Intro "Here you can send a message to..."  
    Section "Your Message" {  
        Input "Subject" @Message.Subject  
        Input "Message" @Message.Message  
        Input "Create listing" ShouldListingBeCreated BOOLEAN  
        ...  
        Action SendMessage (  
            label="Send",  
            logic=@SendMessage(ShouldListingBeCreated),  
            exit (if ShouldListingBeCreated then  
                MessageSentAndListingCreated else MessageSent)  
        )  
    }  
    ...  
}
```

Example of a structured, parseable specification DSL from my past experience, mostly recreated from memory. Note things like references to domain model fields, inheritance from other screens for reusability of patterns, navigation logic.

Em última análise, o MDD nunca decolou para aplicações comerciais, pois se encontra em um nível de abstração inadequado e cria muita sobrecarga e restrições. Mas as LLMs eliminam parte dessa sobrecarga e dessas restrições do MDD, o que traz uma nova esperança de que finalmente possamos nos concentrar em escrever especificações e gerar código a partir delas. Com as LLMs, não estamos mais limitados por uma linguagem de especificação predefinida e analisável, e não precisamos construir geradores de código elaborados. O preço disso é o não determinismo das LLMs, é claro. E a estrutura analisável também tinha vantagens que estamos perdendo agora: podíamos fornecer ao autor da especificação muito suporte de ferramentas para escrever especificações válidas, completas e consistentes. Eu me pergunto se a especificação como fonte, e até mesmo a ancoragem de especificações, podem acabar com as desvantagens tanto do MDD quanto das LLMs: inflexibilidade e não determinismo.

Para ser claro, não estou nostálgico da minha experiência passada com MDD e dizendo "poderíamos muito bem trazer isso de volta". Mas devemos analisar as

tentativas de codificação a partir de especificações no passado para aprendermos com elas quando explorarmos a codificação orientada a especificações hoje.

Conclusões

Na minha experiência pessoal com codificação assistida por IA, também costumo dedicar tempo a elaborar cuidadosamente uma especificação prévia para fornecer ao agente de codificação. Portanto, o princípio geral de "especificar primeiro" é definitivamente valioso em muitas situações, e as diferentes abordagens sobre como estruturar essa especificação são muito procuradas. Elas estão entre as perguntas mais frequentes que ouço atualmente de profissionais da área: "Como estruturo meu banco de memória?", "Como escrevo uma boa especificação e um documento de design para IA?".

Mas o termo "desenvolvimento orientado a especificações" ainda não está muito bem definido e já está semanticamente difuso. Recentemente, inclusive, ouvi pessoas usarem "especificação" basicamente como sinônimo de "instrução detalhada".

Em relação às ferramentas que experimentei, listei aqui muitas das minhas dúvidas sobre a sua utilidade no mundo real. Pergunto-me se algumas delas estão tentando alimentar agentes de IA com nossos fluxos de trabalho existentes de forma muito literal, amplificando, em última análise, desafios já existentes, como sobrecarga de revisões e alucinações. Principalmente com as abordagens mais elaboradas que criam muitos arquivos, não consigo deixar de pensar na palavra composta alemã "Verschlimmbesserung": Será que estamos piorando algo na tentativa de melhorá-lo?